

SPRFST

The Guidebook

Version 0.1.0-beta 04 October 2026

Typeset by SPRFST. No library was used to write this file.

Contents

First steps	3
Values and names	5
Making decisions	7
Repeating work	9
Functions	11
Text	13
Lists	15
Maps and sets	17
Objects	19
Traits	21
Enums and matching	23
When things go wrong	25
Generics	28
Modules and packages	30
Testing	32
Memory	34
Doing several things	37
Files and data	39
The database	41
Drawing	43
Windows	46
The web	50
Learning machines	54
Games	57
Speed	61
The debugger	65
Forge, and extending SPRFST	68
SPRFST Studio	71
The standard library	74
A whole program	78

First steps

SPRFST is a small language. You can read all of it in an afternoon, and the compiler is quick enough that running a program feels like pressing a key rather than starting a build.

Here is a whole program.

```
use std.io

fn main() {
    io.say("hello, world")
}
```

Three things are happening.

`use std.io` brings in the input and output module. Nothing is imported for you behind your back, so you can always tell where a name came from.

`fn main()` declares the function the runner starts with.

`io.say` writes a line.

Save that as `hello.spf` and run it:

```
sprfst run hello.spf
```

There is no build step to remember. `run` compiles and executes in one go; `build` is there for when you want the artefacts.

Making a project

For anything bigger than one file, start a project:

```
sprfst new my-app
cd my-app
sprfst run .
```

That gives you `project.sprfst`, a `src/` folder with `main.spf`, a `tests/` folder and an `assets/` folder. The dot in `sprfst run .` means "the project here" –the runner reads the entry from `project.sprfst`.

Try it

Change the message, then make the program say two lines instead of one.

```
use std.io

fn main() {
    io.say("my name is ...")
    io.say("and this is my first SPRFST program")
}
```

Exercise

Write a program that prints your name, then prints a line of dashes under it. You only need `io.say` and one piece of text.

Values and names

Two words introduce a name: `let` for a value that never changes, and `var` for one that does.

```
use std.io

fn main() {
    let name = "Ada"
    var score = 0

    score = score + 10
    score = score + 5

    io.say("{name} scored {score}")
}
```

Reach for `let` first. Use `var` only when the value really has to change, and the next person reading your code will know which is which at a glance.

The built in types

type	what it holds	example
Int	whole numbers	42, -7, 1_000_000
Num	numbers with a decimal point	3.14, -0.5
Text	words	"hello"
Bool	true or false	true
Byte	a single byte	0x1F
Nil	nothing at all	nil

The compiler works out the type from the value, so you rarely write one down. When you want to be explicit, say so after a colon:

```
use std.io

fn main() {
    let count: Int = 3
    let ratio: Num = 0.75
    let label: Text = "widgets"
    io.say("{count} {label} at {ratio}")
}
```

Putting values into text

Curly braces inside text hold an expression. This is the usual way to build a message.

```
use std.io

fn main() {
    let width = 4
    let height = 7
    io.say("a {width} by {height} room has {width * height} square metres")
}
```

If you need a real brace, double it: `{{` prints `{` .

Numbers

`+` `-` `*` `/` do what you expect. Division always gives a `Num` , so `10 / 4` is `2.5` and never quietly throws the remainder away. `%` gives the remainder and `**` raises to a power.

```
use std.io

fn main() {
    io.say("{10 / 4}")
    io.say("{10 % 4}")
    io.say("{2 ** 10}")
}
```

Exercise

Work out how many seconds there are in a week, using named values for the parts rather than one long sum.

Making decisions

`if` chooses between paths. There are no brackets around the condition.

```
use std.io

fn main() {
    let temperature = 31

    if temperature > 30 {
        io.say("hot")
    } else if temperature > 18 {
        io.say("pleasant")
    } else {
        io.say("cold")
    }
}
```

`if` gives a value

A block ends in a value, so `if` can sit on the right of `let` :

```
use std.io

fn main() {
    let hour = 15
    let greeting = if hour < 12 { "good morning" } else { "good afternoon" }
    io.say(greeting)
}
```

Joining conditions

SPRFST spells the logical operators out: `and` , `or` , `not` . They read aloud the way they behave.

```
use std.io

fn main() {
    let age = 24
    let member = true

    if age >= 18 and member {
        io.say("welcome in")
    }
    if not member or age < 18 {
        io.say("sign up at the desk")
    }
}
```

`and` and `or` stop early: if the left side already decides the answer, the right side is never worked out.

Comparing

`==` `!=` `<` `<=` `>` `>=` work on numbers and text. Text compares alphabetically.

```
use std::io;

fn main() {
    io::say("{ \"apple\" < \"banana\" }")
    io::say("{ 3 != 4 }")
}
```

Exercise

Write a function that takes a year and says whether it is a leap year. A year is a leap year when it divides by 4, except years that divide by 100, unless they also divide by 400.

Repeating work

Three loops, each with a clear job.

``while`` — carry on until something changes

```
use std.io

fn main() {
    var countdown = 3
    while countdown > 0 {
        io.say("{countdown}...")
        countdown = countdown - 1
    }
    io.say("go")
}
```

``for`` — walk through a collection or a range

```
use std.io

fn main() {
    for n in 1..4 {
        io.say("number {n}")
    }

    for colour in ["red", "green", "blue"] {
        io.say(colour)
    }
}
```

`1..4` counts 1, 2, 3. Use `1..=4` when you want to include the end.

``loop`` — go round until you break out

```
use std.io

fn main() {
    var total = 0
    var n = 1
    loop {
        total = total + n
        if total > 20 { break }
        n = n + 1
    }
    io.say("stopped at n = {n}, total = {total}")
}
```

`skip` and `break`

`skip` jumps to the next turn of the loop; `break` leaves it.

```
use std.io

fn main() {
    for n in 1..11 {
        if n % 2 == 0 { skip }
        if n > 7 { break }
        io.say("odd: {n}")
    }
}
```

Walking a map

```
use std.io

fn main() {
    let prices = ["bread": 2, "milk": 1, "jam": 4]
    for item in prices.keys() {
        io.say("{item} costs {prices.get(item) ?? 0}")
    }
}
```

Exercise

Print the first fifteen numbers of the Fibonacci sequence, one per line, using a `while` loop and two `var` values.

Functions

A function takes values in and gives one back. `give` is the word for handing a value back.

```
use std.io

fn area(width: Num, height: Num) -> Num {
    give width * height
}

fn main() {
    io.say("{area(3.0, 4.0)}")
}
```

Parameter types are always written down. The result type goes after `->` ; leave it off when the function gives nothing back.

Short functions

When the whole body is one expression, use `=>` :

```
use std.io

fn double(n: Int) -> Int => n * 2
fn shout(words: Text) -> Text => words.upper() + "!"

fn main() {
    io.say("{double(21)}")
    io.say(shout("quietly"))
}
```

Functions as values

A function can be stored, passed and returned.

```
use std.io

fn apply_twice(value: Int, change: fn(Int) -> Int) -> Int {
    give change(change(value))
}

fn main() {
    let add_three = fn(n: Int) => n + 3
    io.say("{apply_twice(10, add_three)}")
}
```

A function written inline like that is a lambda. It can see the values around it:

```
use std.io

fn main() {
    let tax = 0.2
    let with_tax = fn(price: Num) => price * (1.0 + tax)
    io.say("{with_tax(100.0)}")
}
```

Lambdas capture by value, so `tax` is fixed at the moment the lambda is made. Nothing changes under you later.

Pipelines

`|>` sends the value on the left into the function on the right. It turns a nest of calls inside out.

```
use std.io

fn main() {
    let total = [1, 2, 3, 4, 5] |> sum
    io.say("{total}")
    io.say("{ " hello " |> trim |> upper}")
}
```

Exercise

Write `celsius_to_fahrenheit` and `fahrenheit_to_celsius`, then print a small conversion table from -10 to 40 in steps of 10.

Text

Text in SPRFST is a sequence of bytes holding UTF-8. The methods you reach for most are all on the value itself.

```
use std.io

fn main() {
    let line = "  SPRFST is small and quick  "
    io.say("[{line.trim()}]")
    io.say("{line.trim().upper()}")
    io.say("{line.trim().len()} bytes")
    io.say("{line.contains(\"quick\")}")
    io.say("{line.trim().replace(\"quick\", \"fast\")}")
}
```

Splitting and joining

```
use std.io

fn main() {
    let csv = "ada,grace,alan"
    let names = csv.split(",")
    io.say("{names}")
    io.say("{names.join(\" and \")}")
    io.say("first is {names.first() ?? \"nobody\"}")
}
```

Slicing and padding

```
use std.io

fn main() {
    let word = "guidebook"
    io.say(word.slice(0, 5))
    io.say(word.reverse())
    io.say("[{\"42\".pad_left(6, \"0\")}]")
    io.say("[{\"name\".pad_right(10)}]")
}
```

Characters

`chars()` gives a list of one character pieces.

```
use std.io

fn main() {
    var vowels = 0
    for ch in "programming language".chars() {
```

```
        if "aeiou".contains(ch) { vowels = vowels + 1 }
    }
    io.say("{vowels} vowels")
}
```

Turning text into numbers

`to_int` and `to_num` give back nothing when the text is not a number, so you have to say what should happen instead.

```
use std.io

fn main() {
    io.say("{\"42\".to_int() ?? 0}")
    io.say("{\"not a number\".to_int() ?? 0}")
}
```

Exercise

Write `is_palindrome(text)` that ignores spaces and capital letters. `"Never odd or even"` should come back `true`.

Lists

A list holds values of one type, in order.

```
use std.io

fn main() {
    var scores = [7, 3, 9, 1]
    scores.push(5)
    io.say("{scores}")
    io.say("{scores.len()} items")
    io.say("first {scores.first() ?? 0}, last {scores.last() ?? 0}")
    io.say("item 2 is {scores[2]}")
}
```

Shaping a list

`map`, `filter` and `reduce` make a new list rather than changing the old one.

```
use std.io

fn main() {
    let numbers = [1, 2, 3, 4, 5, 6]
    io.say("{numbers.map(fn(n) => n * n)}")
    io.say("{numbers.filter(fn(n) => n % 2 == 0)}")
    io.say("{numbers.reduce(0, fn(total, n) => total + n)}")
}
```

Asking questions

```
use std.io

fn main() {
    let words = ["apple", "fig", "cherry"]
    io.say("{words.any(fn(w) => w.len() > 5)}")
    io.say("{words.all(fn(w) => w.len() > 2)}")
    io.say("{words.count(fn(w) => w.contains(\"e\"))}")
    io.say("{words.find(fn(w) => w.starts_with(\"f\")) ?? \"none\"}")
}
```

Sorting

`sort` works on anything orderable; `sort_by` takes a comparison that answers "does a come before b".

```
use std.io

fn main() {
    let names = ["Grace", "ada", "Alan"]
    io.say("{names.sort()}")
}
```

```
    io.say("{names.sort_by(fn(a, b) => a.lower() < b.lower())}")
}
```

Sorting is stable, so equal items keep the order you gave them.

Slices

```
use std::io;

fn main() {
    let row = [10, 20, 30, 40, 50]
    io.say("{row.take(2)}")
    io.say("{row.drop(3)}")
    io.say("{row.slice(1, 4)}")
    io.say("{row.reverse()}")
}
```

Exercise

Given a list of numbers, produce a list of running totals: `[1, 2, 3]` becomes `[1, 3, 6]` .

Maps and sets

A map pairs keys with values. The literal uses the same square brackets as a list, with a colon between key and value.

```
use std.io

fn main() {
    var stock = ["apples": 12, "pears": 3]
    stock.set("plums", 7)

    io.say("{stock.get("apples") ?? 0} apples")
    io.say("has pears: {stock.has("pears")}")
    io.say("{stock.len()} kinds of fruit")
    io.say("keys {stock.keys()}")
}
```

`get` gives back "the value, or nothing", which is why `?? 0` is there: it says what to use when the key is missing. The compiler will not let you forget.

An empty map is written `[:]` so it cannot be confused with an empty list.

Walking a map

```
use std.io

fn main() {
    let ages = ["ada": 36, "alan": 41, "grace": 45]
    for name in ages.keys() {
        io.say("{name.pad_right(8)} {ages.get(name) ?? 0}")
    }
}
```

Counting things

A map is the natural way to count.

```
use std.io

fn main() {
    var counts: Map<Text, Int> = [:]
    for word in "the cat sat on the mat the end".split(" ") {
        counts.set(word, (counts.get(word) ?? 0) + 1)
    }
    for word in counts.keys() {
        io.say("{word}: {counts.get(word) ?? 0}")
    }
}
```

Sets

A set holds each value once and answers "is this in here" quickly.

```
use std::io;

fn main() {
    var seen = set([1, 2, 3, 2, 1])
    seen.add(9)
    io.say("{seen.len()} unique values")
    io.say("has 9: {seen.has(9)}")
    io.say("as a list: {seen.items()}")
}
```

Exercise

Read a sentence and report the three most common words. `sort_by` on the key list will help.

Objects

An `object` groups data with the functions that work on it. Fields can have defaults, so making one is short.

```
use std.io

object Counter {
  name: Text = "counter"
  value: Int = 0

  fn bump(self) {
    self.value = self.value + 1
  }

  fn report(self) -> Text => "{self.name} is at {self.value}"
}

fn main() {
  var hits = Counter { name: "page views" }
  hits.bump()
  hits.bump()
  io.say(hits.report())
}
```

Every method that touches the object takes `self` as its first parameter. Writing it out keeps it obvious which functions belong to the instance.

`data` for plain values

When a type is only a bag of fields, use `data`. It has no methods and compares by value.

```
use std.io

data Point {
  x: Num
  y: Num
}

fn main() {
  let a = Point { x: 1.0, y: 2.0 }
  let b = Point { x: 1.0, y: 2.0 }
  io.say("{a.x}, {a.y}")
  io.say("same: {a == b}")
}
```

Composition first

Rather than building deep hierarchies, put one object inside another.

```

use std.io

object Engine {
  power: Int = 90
  fn describe(self) -> Text => "{self.power} hp"
}

object Car {
  make: Text = "saloon"
  engine: Engine = Engine { }
  fn describe(self) -> Text => "{self.make}, {self.engine.describe()}"
}

fn main() {
  let car = Car { make: "estate", engine: Engine { power: 140 } }
  io.say(car.describe())
}

```

Extending

`extends` is there when you genuinely want a specialised version. The child gets the parent's fields and can replace its methods.

```

use std.io

object Shape {
  name: Text = "shape"
  fn area(self) -> Num => 0.0
  fn describe(self) -> Text => "{self.name} with area {self.area()}"
}

object Square extends Shape {
  side: Num = 1.0
  fn area(self) -> Num => self.side * self.side
}

fn main() {
  let s = Square { name: "square", side: 3.0 }
  io.say(s.describe())
}

```

Exercise

Model a bank account with a balance, `deposit`, `withdraw` and a method that refuses to go below zero.

Traits

A trait is a promise: "a type with this trait has these methods". It lets you write one function that works on several types.

```
use std.io

trait Greeter {
    fn name(self) -> Text
    fn greet(self) -> Text => "hello, {self.name()}"
}

object Person {
    given: Text = "friend"
    fn name(self) -> Text => self.given
}

object Robot {
    serial: Int = 1
    fn name(self) -> Text => "unit {self.serial}"
}

impl Greeter for Person {}
impl Greeter for Robot {}

fn main() {
    let crowd = [Person { given: "Ada" }, Robot { serial: 7 }]
    for one in crowd {
        io.say(one.greet())
    }
}
```

Two things worth noticing.

`greet` has a body in the trait. That is a default: any type that takes the trait gets it free, and can replace it if it wants to.

`name` has no body, so each type must supply one. The compiler checks this and tells you exactly which method is missing.

Using a trait as a type

```
use std.io

trait Shape {
    fn area(self) -> Num
}

object Circle {
    radius: Num = 1.0
    fn area(self) -> Num => 3.14159 * self.radius * self.radius
}
```

```

object Rect {
  w: Num = 1.0
  h: Num = 1.0
  fn area(self) -> Num => self.w * self.h
}

impl Shape for Circle {}
impl Shape for Rect {}

fn total(shapes: [Shape]) -> Num {
  var sum = 0.0
  for s in shapes { sum = sum + s.area() }
  give sum
}

fn main() {
  io.say("{total([Circle { radius: 1.0 }, Rect { w: 2.0, h: 3.0 }])}")
}

```

Built in traits

A few names are understood by the compiler: `Show` for values that can be printed, `Eq` for values that compare, `Ord` for values that sort, `Hash` for values that can be map keys, and `Num` for numbers.

Exercise

Add a `perimeter` method to the `Shape` trait with a sensible default of `0.0`, implement it properly for `Rect`, and print both figures.

Enums and matching

An `enum` lists the shapes a value is allowed to take. Each shape may carry data of its own.

```
use std.io

enum Shape {
    Circle(Num)
    Rect(Num, Num)
    Empty
}

fn area(s: Shape) -> Num {
    give match s {
        when Circle(r) -> 3.14159 * r * r
        when Rect(w, h) -> w * h
        when Empty -> 0.0
    }
}

fn main() {
    io.say("{area(Circle(2.0))}")
    io.say("{area(Rect(3.0, 4.0))}")
    io.say("{area(Empty)}")
}
```

`match` pulls the data out as it chooses the arm. The compiler checks that every shape is covered, and names the ones you forgot.

Matching on anything

`match` is not only for enums.

```
use std.io

fn describe(n: Int) -> Text {
    give match n {
        when 0 -> "nothing"
        when 1 -> "one"
        when 2 -> "a pair"
        else -> if n < 0 { "below zero" } else { "many" }
    }
}

fn main() {
    for n in [-3, 0, 1, 2, 17] {
        io.say("{n} is {describe(n)}")
    }
}
```

Guards

`if` after a pattern adds a condition.

```
use std.io

enum Reading { Temperature(Num) Pressure(Num) }

fn comment(r: Reading) -> Text {
    give match r {
        when Temperature(t) if t > 38.0 -> "fever"
        when Temperature(t) -> "temperature {t} is fine"
        when Pressure(p) if p < 90.0 -> "pressure is low"
        when Pressure(p) -> "pressure {p} is fine"
    }
}

fn main() {
    io.say(comment(Temperature(39.4)))
    io.say(comment(Pressure(120.0)))
}
```

List patterns

```
use std.io

fn summarise(items: [Int]) -> Text {
    give match items {
        when [] -> "empty"
        when [one] -> "just {one}"
        when [first, second] -> "{first} and {second}"
        when [first, ..rest] -> "{first} then {rest.len()} more"
        else -> "something else"
    }
}

fn main() {
    io.say(summarise([]))
    io.say(summarise([5]))
    io.say(summarise([1, 2]))
    io.say(summarise([1, 2, 3, 4]))
}
```

Exercise

Model a simple calculator: an enum with `Add`, `Subtract`, `Multiply` and `Divide`, each carrying two numbers, and a function that works out the answer.

When things go wrong

SPRFST separates two different ideas: a value that might be missing, and an operation that might fail.

Might be missing: `T?`

A question mark means "or nothing".

```
use std.io

fn main() {
    let ages = ["ada": 36]
    let found = ages.get("ada")      ~~ this is Int?
    let missing = ages.get("nobody")

    io.say("{found ?? 0}")
    io.say("{missing ?? 0}")
}
```

`??` supplies a value for the nothing case. The compiler will not let you use an `Int?` where an `Int` is wanted, so a missing value can never slip through unnoticed.

`?.` calls a method only when there is something there:

```
use std.io

fn first_word(line: Text?) -> Text {
    give line?.split(" ").first() ?? "nothing"
}

fn main() {
    io.say(first_word("hello there"))
    io.say(first_word(nil))
}
```

Might fail: `Result<T>`

A function that can fail gives back `Ok(value)` or `Err(reason)`.

```
use std.io

fn parse_age(text: Text) -> Result<Int> {
    let n = text.to_int()
    if n == nil { give Err("'{}' is not a number") }
    let age = n ?? 0
    if age < 0 { give Err("an age cannot be negative") }
    give Ok(age)
}

fn main() {
    for input in ["34", "abc", "-2"] {
```

```

        match parse_age(input) {
            when Ok(age) -> io.say("{input}: age {age}")
            when Err(why) -> io.say("{input}: {why}")
        }
    }
}

```

Stopping on the spot

Sometimes the right answer is "this should never happen". `fail` raises a problem that travels up until something catches it.

```

use std.io

fn divide(a: Int, b: Int) -> Num {
    if b == 0 { fail "divide by zero" }
    give a / b
}

fn main() {
    io.say("{divide(10, 2)}")
    try {
        io.say("{divide(1, 0)}")
    } catch problem {
        io.say("caught: {problem}")
    }
    io.say("carrying on")
}

```

Use `Err` for problems the caller should think about, and `fail` for mistakes in the program itself.

Cleaning up with `defer`

`defer` runs when the function leaves, whichever way it leaves.

```

use std.io

fn work() {
    defer io.say(" closed")
    io.say(" opened")
    io.say(" working")
}

fn main() {
    io.say("start")
    work()
    io.say("done")
}

```

Exercise

Write `safe_sqrt(n)` that gives `Err` for negative input, then use it on a list of numbers and print one line per result.

Generics

A generic function works for many types without giving up type checking.

```
use std.io

fn first_of<T>(items: [T]) -> T? {
    if items.is_empty() { give nil }
    give items[0]
}

fn main() {
    io.say("{first_of([3, 4, 5]) ?? 0}")
    io.say("{first_of([\"x\", \"y\"]) ?? \"none\"}")
}
```

`T` stands for whatever type the caller passes. Inside the function you can only do things that work for every possible `T`, which is what makes the checking sound.

Asking for more

A bound after the colon says what `T` must be able to do.

```
use std.io

fn larger<T: Ord>(a: T, b: T) -> T {
    if a > b { give a }
    give b
}

fn main() {
    io.say("{larger(3, 9)}")
    io.say("{larger(\"apple\", \"pear\")}")
}
```

Without `: Ord`, the compiler would refuse the `>` and tell you to add the bound.

Generic types

```
use std.io

object Stack<T> {
    items: [T] = []

    fn add(self, item: T) { self.items.push(item) }
    fn take(self) -> T? => self.items.pop()
    fn size(self) -> Int => self.items.len()
}

fn main() {
    var numbers = Stack<Int> { }
```

```

numbers.add(1)
numbers.add(2)
io.say("size {numbers.size()}, took {numbers.take() ?? 0}")

var words = Stack<Text> { }
words.add("hello")
io.say("took {words.take() ?? ""}")
}

```

Two parameters

```

use std.io

data Pair<A, B> {
    left: A
    right: B
}

fn main() {
    let answer = Pair<Text, Int> { left: "answer", right: 42 }
    io.say("{answer.left} = {answer.right}")
}

```

How it works underneath

Generics are erased: one compiled copy of the function serves every type, and the runtime carries the type information it needs on the values themselves. Compile times stay flat however many types you use.

Exercise

Write a generic `swap(pair)` that returns a new `Pair` with the two sides exchanged.

Modules and packages

Every file is a module. `use` brings another one into view.

```
src/  
  main.spf  
  shapes.spf
```

`shapes.spf` :

```
module shapes  
  
pub fn area_of_square(side: Num) -> Num => side * side  
  
fn secret_helper() -> Int => 42
```

`main.spf` :

```
use std.io  
use shapes  
  
fn main() {  
  io.say("{shapes.area_of_square(3.0)}")  
}
```

`pub` marks what the outside world may use. Everything else stays private to the file, so you can rearrange the inside freely.

The standard library

Modules under `std` come with the language:

`io` `math` `fs` `path` `time` `rand` `sys` `json` `csv` `hash` `net` `http` `thread` `task` `db` `tensor` `ui` `draw`, plus `core`, `collections`, `testing` and `ai` which are written in SPRFST itself.

```
use std.math  
use std.io  
  
fn main() {  
  io.say("{math.sqrt(144.0)}")  
  io.say("{math.clamp(42.0, 0.0, 10.0)}")  
}
```

Forge, the package manager

Forge has no central registry. A package is a folder with a `project.sprfst` and some source. You add one by path or by git URL:

```
sprfst add maths --path=../maths
sprfst add colour --git=https://example.com/colour.git
sprfst install
sprfst remove colour
```

`add` copies the package into `packages/`, works out a digest of its contents and writes it to `forge.lock`. `install` re-fetches everything in the lockfile and refuses to continue if a digest has changed, so a package cannot be swapped underneath you.

To share your own work:

```
sprfst package
```

That writes `name-version.forge`, a single file with every source and a digest of the whole thing.

Exercise

Split a program you have already written into two files, with the calculations in their own module and only the parts you need marked `pub`.

Testing

Tests live beside the code, in the same file or in `tests/`.

```
use std.io
use std.testing

fn double(n: Int) -> Int => n * 2

test "double doubles" {
  testing.same(double(2), 4, "two")
  testing.same(double(0), 0, "zero")
  testing.same(double(0 - 3), 0 - 6, "negative")
}

fn main() {
  io.say("{double(21)}")
}
```

Run them with:

```
sprfst test
```

Every `test` block runs on its own, and a failure reports which assertion gave way and what it saw instead.

What `std.testing` offers

- `same(got, wanted, what)` —fails unless they match
- `different(got, unwanted, what)`
- `yes(condition, what)` and `no(condition, what)`
- `close(got, wanted, tolerance, what)` —for numbers with decimals
- `holds(list, item, what)`
- `failed(result, what)` —expects an `Err`

```
use std.testing
use std.io

fn average(xs: [Num]) -> Num {
  if xs.is_empty() { give 0.0 }
  give xs.sum() / xs.len()
}

test "average" {
  testing.close(average([2.0, 4.0]), 3.0, 0.0001, "two numbers")
  testing.close(average([]), 0.0, 0.0001, "no numbers")
}

fn main() { io.say("{average([1.0, 2.0, 3.0])}") }
```

Benchmarks

`bench` blocks are timed rather than checked. The runner repeats the body until it has a steady figure and reports operations per second.

```
use std.io

bench "building a list" {
    var xs: [Int] = []
    var i = 0
    while i < 500 {
        xs.push(i)
        i = i + 1
    }
}

fn main() { io.say("run me with sprfst test") }
```

Exercise

Take the palindrome function from chapter six and write tests for the empty string, a single letter, a real palindrome and a non-palindrome.

Memory

Most of the time you do not think about memory in SPRFST. Values live where they need to and the runtime tidies up. This chapter is about what is actually happening, and the words you can use when you want control.

Where values live

Small values —`Int`, `Num`, `Bool`, `Byte` —live on the stack, inside the frame of the function using them. Nothing is allocated.

Bigger values —text, lists, maps, objects —live on the heap, and the variable holds a reference to them. Passing a list to a function passes the reference, not a copy, so it is cheap however long the list is.

```
use std.io

fn add_one(items: [Int]) {
    items.push(1)      ~~ the caller sees this
}

fn main() {
    var numbers = [0]
    add_one(numbers)
    io.say("{numbers}")
}
```

When you want a copy, ask for one:

```
use std.io

fn main() {
    let original = [1, 2, 3]
    var copy = clone(original)
    copy.push(4)
    io.say("original {original}")
    io.say("copy      {copy}")
}
```

Clearing up

The runtime collects values nothing refers to any more. Collection happens between statements, never in the middle of one, and it walks only live data. You can see the figures:

```
sprfst run main.spf --time
```

which reports instructions executed, peak memory and how many times the collector ran.

Ownership, when you want it

`own T` says a value has exactly one owner. Handing it to something else that wants to own it **moves** it, and the compiler stops you from using the old name afterwards.

```
use std.io

fn peek(items: ref [Int]) -> Int => items.len()
fn consume(items: own [Int]) -> Int => items.len()

fn main() {
    let a: own [Int] = [1, 2, 3]

    io.say("borrowing is fine: {peek(a)}")
    io.say("and `a` still works: {a.len()}")

    let b: own [Int] = a          ~~ ownership handed over
    io.say("`b` owns it now: {consume(b)}")

    let c: own [Int] = [9, 9]
    io.say("a copy keeps the original: {consume(clone(c))} and {c.len()}")
}
```

Use the moved name and the compiler says so, with the place it went:

```
error[E0301]  `a` was moved away
+- main.spf:6:22
|
6 |         io.say("{consume(a)}")
|                   - used after the move
5 |         let b: own [Int] = a
|                   - ownership moved here
|
+ note  `own` values have exactly one owner at a time
+ fixes
  • clone it first:  let copy = clone(a)
  • or borrow it instead:  fn use(x: ref List<Int>)
```

`ref T` borrows instead: the caller keeps ownership and the callee may read. `ref mut T` borrows for writing.

Ownership is opt in, and the check is deliberately simple—a name is treated as moved from the line the move appears on, so moving inside a branch counts for everything after it. Reach for `own` on large buffers, on handles that must be closed exactly once, and anywhere a second owner would be a bug. Leave it off and the collector handles everything, as it does in every other chapter.

Cleaning up at the right moment

`defer` runs a block when the function ends, however it ends—normally, through `give`, or through an error on the way out.

```

use std.io
use std.fs

fn work() {
    fs.write("scratch.txt", "hello")
    defer {
        fs.remove("scratch.txt")
        io.say("tidied up")
    }
    io.say("while we work the file exists: {fs.exists("scratch.txt")}")
}

fn main() {
    work()
    io.say("afterwards: {fs.exists("scratch.txt")}")
}

```

This is the tool to reach for when something must be released: a file, a lock, a socket, a database handle. An object may also declare a `drop` method, which the language reserves for the same purpose, but the runtime does not call it yet—today, `defer` is what actually runs, and the examples use it.

`unsafe`

An `unsafe` block lets you do things the checker cannot verify, such as talking to C. It is deliberately ugly so it is easy to find.

```

unsafe {
    ~~ raw pointer work goes here
}

```

Exercise

Write a function that takes a list, makes a sorted copy and returns it, leaving the original untouched. Prove it with a print before and after.

Doing several things

Tasks and futures

Mark a function `task` when it is meant to run alongside other work. `spawn` starts it and gives back a future; `await` collects the answer.

```
use std.io

task fetch(id: Int) -> Int {
    give id * 100
}

fn main() {
    let a = spawn fetch(1)
    let b = spawn fetch(2)
    let c = spawn fetch(3)

    io.say("{await a} {await b} {await c}")
}
```

Spawning is cheap: nothing runs until something awaits it or the scheduler gets a turn. That keeps the common "start several, collect them all" pattern fast.

Channels

A channel is a queue that one part of the program writes to and another reads from.

```
use std.io
use std.thread

fn main() {
    let jobs = thread.channel(8)

    for n in 1..5 {
        jobs.send(n * n)
    }

    io.say("{jobs.len()} jobs waiting")
    while jobs.len() > 0 {
        io.say("took {jobs.recv() ?? 0}")
    }
}
```

`recv` gives nothing when the channel is empty and closed, so a loop can tell the difference between "wait" and "finished".

Locks and atomics

When two parts of the program really do share one value, guard it.

```
use std.io
use std.thread

fn main() {
    let guard = thread.lock()
    let counter = thread.counter(0)

    thread.acquire(guard)
    thread.atomic_add(counter, 5)
    thread.atomic_add(counter, 3)
    thread.release(guard)

    io.say("counter is {thread.atomic_add(counter, 0)}")
    io.say("{thread.cpus()} cores available")
}
```

What the runtime actually does

Threads are real operating system threads, but the interpreter holds one lock while it runs, handing it over every so often and whenever a task blocks on input, output or the network. That means input and output overlap properly and the program stays responsive, while two pieces of SPRFST code never run at the same instant. It is honest cooperative scheduling rather than parallel execution, and it keeps the memory model simple: you cannot see a half written value.

Exercise

Spawn five tasks that each square a number, collect them into a list, and print the total.

Files and data

Reading and writing

```
use std.io
use std.fs
use std.path

fn main() {
    let folder = "/tmp/sprfst-chapter-18"
    fs.mkdir(folder)

    let note = path.join(folder, "note.txt")
    fs.write(note, "first line\n")
    fs.append(note, "second line\n")

    io.say("{fs.size(note)} bytes")
    io.say(fs.read(note) ?? "")
    io.say("exists: {fs.exists(note)}")
    io.say("here: {fs.list(folder)}")

    fs.remove(note)
}
```

`fs.read` gives `Text?` because the file may not be there. The compiler makes you decide what happens then.

JSON

```
use std.io
use std.json

fn main() {
    let record = ["name": "Ada", "age": 36, "tags": ["maths", "engines"]]

    let compact = json.write(record)
    io.say(compact)
    io.say(json.pretty(record))

    let back = json.parse(compact)
    io.say("name is {back?.get("name")} ?? "?"")
}
```

CSV

```
use std.io
use std.csv

fn main() {
```

```

let table = "city,people\nhouston,2300000\naustin,960000"
for row in csv.parse(table) {
    io.say("{row}")
}

let rows = [{"name", "score"}, {"ada", "99"}, {"alan", "95"}]
io.say(csv.write(rows))
}

```

Paths

```

use std.io
use std.path

fn main() {
    let p = "/home/user/project/src/main.spf"
    io.say(path.dirname(p))
    io.say(path.basename(p))
    io.say(path.ext(p))
    io.say(path.join("src", "main.spf"))
}

```

Hashing and encoding

```

use std.io
use std.hash

fn main() {
    io.say(hash.sha256("hello"))
    io.say("{hash.crc32("hello")}")
    let packed = hash.base64("SPRFST")
    io.say(packed)
    io.say(hash.unbase64(packed) ?? "")
}

```

Exercise

Write a program that reads a CSV file of names and scores, works out the average, and writes a JSON summary next to it.

The database

SPRFST carries a small relational database called Ember. There is no server to install and no driver to add: `use std.db` and you have tables, SQL and transactions.

```
use std.io
use std.db

fn main() {
    let store = db.open(":memory:")

    db.exec(store, "create table people (id int, name text, city text, score num)")
    db.exec(store, "insert into people values (1, 'Ada', 'London', 99.5)")
    db.exec(store, "insert into people values (2, 'Grace', 'New York', 97.0)")
    db.exec(store, "insert into people values (3, 'Alan', 'London', 95.5)")

    for row in db.query(store, "select name, city from people order by name") {
        io.say("{row.get(\"name\") ?? \"?\"} - {row.get(\"city\") ?? \"?\"}")
    }

    db.close(store)
}
```

`:memory:` keeps everything in memory. Give a file path instead and Ember writes through on every change, so the data is there next time.

Querying

```
use std.io
use std.db

fn main() {
    let store = db.open(":memory:")
    db.exec(store, "create table fruit (name text, crates int)")
    db.exec(store, "insert into fruit values ('apples', 12), ('pears', 4), ('plums', 0)")

    let in_stock = db.query(store, "select name from fruit where crates > 0 order by crates desc")
    io.say("{in_stock}")

    let counted = db.query(store, "select count(*) from fruit")
    io.say("{counted.first()?.get(\"count\") ?? 0} kinds")

    let like = db.query(store, "select name from fruit where name like 'p%'")
    io.say("{like}")

    db.close(store)
}
```

Ember understands `create table`, `drop table`, `insert`, `select` with `where`, `order by` and `limit`, `update` and `delete`. Conditions support `=` `!=` `<` `>` `<=` `>=` and `like`, joined with `and` and `or`

Transactions

```
use std.io
use std.db

fn main() {
    let store = db.open(":memory:")
    db.exec(store, "create table t (n int)")
    db.exec(store, "insert into t values (1), (2), (3)")

    db.begin(store)
    db.exec(store, "delete from t")
    io.say("inside: {db.query(store, "select count(*) from t").first()?.get("count") ?
? 0}")
    db.rollback(store)
    io.say("after rollback: {db.query(store, "select count(*) from t").first()?.get("c
ount") ?? 0}")

    db.close(store)
}
```

`db.begin` takes a snapshot. `db.commit` keeps the changes and writes them out; `db.rollback` puts the snapshot back.

Exercise

Build a tiny address book: a table, a function to add a person, a function to search by the start of a name, and a count of the total.

Drawing

`std.draw` is a canvas of pixels, a handful of shapes and a PNG writer. There is no graphics card involved and nothing to install: you fill a buffer and save it.

A canvas and some shapes

```
use std.io
use std.draw

fn main() {
    let canvas = draw.canvas(320, 200)
    draw.clear(canvas, draw.rgb(11, 11, 13))

    draw.rect(canvas, 24, 24, 130, 56, draw.rgb(255, 161, 54))
    draw.text(canvas, 36, 44, "SPRFST", draw.rgb(11, 11, 13))
    draw.circle(canvas, 250, 120, 40, draw.rgb(120, 200, 255))
    draw.line(canvas, 0, 199, 319, 0, draw.rgb(48, 44, 40))

    if draw.save_png(canvas, "shapes.png") {
        io.say("wrote shapes.png at {draw.size(canvas)}")
    } else {
        io.say("the file could not be written")
    }
}
```

Colours are made with `draw.rgb(red, green, blue)`, each part from 0 to 255. Coordinates start at the top left corner, `x` to the right and `y` downwards. Everything is clipped to the canvas, so drawing off the edge is harmless.

A chart, because charts are useful

Nothing here is special to charts—it is arithmetic and rectangles.

```
use std.io
use std.draw
use std.math

fn chart(values: [Num], file: Text) -> Bool {
    let w = 420
    let h = 220
    let canvas = draw.canvas(w, h)
    draw.clear(canvas, draw.rgb(11, 11, 13))

    let top = values.max() ?? 1.0
    let gap = 10
    let bar = math.floor((w - gap) / values.len()) - gap

    for i in 0..values.len() {
        let tall = math.round(values[i] / top * (h - 50))
        draw.rect(canvas, gap + i * (bar + gap), h - tall - 24, bar, tall,
```

```

        draw.rgb(255, 161, 54))
    }

    draw.line(canvas, 0, h - 24, w, h - 24, draw.rgb(80, 76, 70))
    draw.text(canvas, 12, 16, "peak {top}", draw.rgb(180, 176, 170))
    give draw.save_png(canvas, file)
}

fn main() {
    io.say("chart written: {chart([3.0, 8.0, 5.5, 9.0, 2.0, 6.5], "chart.png")}")
}

```

Pixel by pixel

`draw.pixel` is the lowest level there is, and it is fast enough for real work: a 256 by 256 gradient is sixty five thousand calls and takes a few milliseconds.

```

use std.io
use std.draw
use std.math

fn main() {
    let size = 160
    let canvas = draw.canvas(size, size)

    var y = 0
    while y < size {
        var x = 0
        while x < size {
            let r = math.floor(x * 255 / size)
            let g = math.floor((x + y) * 128 / size)
            let b = math.floor(y * 255 / size)
            draw.pixel(canvas, x, y, draw.rgb(r, g, b))
            x = x + 1
        }
        y = y + 1
    }

    draw.save_png(canvas, "gradient.png")
    io.say("gradient {draw.size(canvas)} done")
}

```

The logo is a SPRFST program

`assets/logo/make_icons.spf` in this repository draws the SPRFST mark with nothing but `std.draw`, then writes it out at every size the app bundle needs:

```

sprfst run assets/logo/make_icons.spf

```

The icons you see in Studio and in the Dock came out of that file. If you want to change the mark, change the program.

Exercise

Draw a clock face: a circle, twelve ticks around the edge worked out with `math.sin` and `math.cos`, and two lines for the hands at the time `time.parts(time.now())` gives you.

Windows

A desktop program in SPRFST is an `app` . You describe what the window contains and what each control does; the runtime keeps the two in step.

The whole shape of an app

```
app "Counter" {
  var count = 0
  var step = 1

  window {
    title: "SPRFST Counter"
    width: 420
    height: 280

    title "Count is {count}"
    divider
    button "Add {step}" {
      on click {
        count = count + step
      }
    }
    button "Bigger steps" {
      on click {
        step = step * 2
      }
    }
    button "Reset" {
      on click {
        count = 0
        step = 1
      }
    }
  }
}
```

Three things are going on.

`var count = 0` inside the `app` is **state** . It lives as long as the window does, and any part of the app may change it.

A label with a `{hole}` in it is **bound** . When the state inside the hole changes, that label is redrawn—you never refresh anything by hand.

`on click { ... }` is ordinary SPRFST code. It can call functions, read files, start tasks. There is no separate event language.

The elements

Element	What it is	Properties
window	the frame everything sits in	title, width, height
title / heading	large text	the label
text	ordinary text	the label
button	a button	on click
field / input	one line of typing	value, on change
toggle / check	a tick box	value, on change
list	a list of values	items
divider	a rule between things	—

Reading what the person typed

An event can carry a value. Name it in brackets after the event and it arrives as an ordinary parameter:

```
app "Notes" {
  var draft = ""
  var notes: [Text] = []
  var pinned = false

  window {
    title: "Notes"
    width: 460
    height: 420

    title "{notes.len()} notes"
    field "New note" {
      value: draft
      on change(text) {
        draft = text
      }
    }
    button "Keep it" {
      on click {
        if draft.trim().len() > 0 {
          notes.push(draft.trim())
          draft = ""
        }
      }
    }
  }
  toggle "Pin this window" {
    value: pinned
    on change(ticked) {
      pinned = ticked
    }
  }
  divider
```

```

        list {
            items: notes
        }
    }
}

```

`value: draft` sends state *into* the control, `on change(text)` brings the answer back out. Clearing `draft` in the button handler empties the box, because the binding runs again after every event.

Where the window comes from

The same program runs against more than one backend, chosen with the `SPRFST_UI` environment variable:

Value	What you get
<code>term</code>	the window drawn in the terminal, with numbered controls
<code>json</code>	one line of JSON describing the window, then exit
<code>none</code>	build the window, print its description, exit
<code>unset</code>	<code>term</code> when the terminal is interactive, otherwise <code>none</code>

On a Mac, and inside SPRFST Studio, the app is hosted in a real AppKit window. The terminal backend exists so an interface can be driven on a machine with no screen—which is how every window in this guidebook is tested.

```
SPRFST_UI=term sprfst run examples/18-gui-counter.spf
```

Building an interface by hand

`app` is sugar over `std.ui`, which you can call directly when the interface is generated rather than written out. `ui.window` gives you a node id, `ui.node` adds children, and `ui.on` attaches handlers.

```

use std.io
use std.ui

var clicks = 0

fn main() {
    let w = ui.window("Built by hand", 360, 200)
    let label = ui.node(w, "title", "0 clicks")
    let press = ui.node(w, "button", "Click me")

    ui.on(press, "click", fn() => {
        clicks = clicks + 1
        ui.set(label, "text", "{clicks} clicks")
    })

    ~~ `ui.emit` fires a handler from code, which is how you test an
    ~~ interface without a person pressing anything

```

```
    ui.emit(press, "click")
    ui.emit(press, "click")
    io.say("the label now says: {ui.get(label, "text")}")
    io.say("backend: {ui.backend()}")

    ui.run()
}
```

Note `var clicks` outside `main`. A closure captures the values around it as they were, so a counter that has to survive between clicks lives in app state or in a module level `var`, not in a local.

Exercise

Add a "Remove last" button to the notes app, and a heading that shows the most recent note, or `nothing yet` when the list is empty.

The web

`std.http` has two halves: fetching things, and serving them. Both are built on `std.net`, which is plain sockets if you ever need to go lower.

Serving

A server is a port and a function. The function is handed a request and gives back a reply.

```
use std.io
use std.http

fn handle(request: Map<Text, Any>) -> Map<Text, Any> {
    give ["status": 200, "type": "text/plain", "body": "hello from SPRFST"]
}

fn main() {
    io.say("listening on http://localhost:8080")
    http.serve(8080, handle)
}
```

`http.serve` does not come back—it accepts connections until the program is stopped, or until some other part of the program calls `http.stop()`.

What is in a request

Key	Example
method	GET
path	/notes/3
query	tag=work&sort=new
headers	a map of lowercase header names to values
body	the request body as text

What a reply may contain

Key	Default	Meaning
status	200	the status code
type	text/html; charset=utf-8	the content type
body	empty	the text to send

A plain `Text` reply also works, and is treated as HTML with status 200.

Routing is just code

Because the handler is an ordinary function, you can call it directly—which means a web application can be tested without a socket in sight.

```
use std.io
use std.json

fn handle(request: Map<Text, Any>) -> Map<Text, Any> {
    let path = request.get("path") ?? "/"
    let method = request.get("method") ?? "GET"

    if path == "/" {
        give ["status": 200, "body": "<h1>SPRFST</h1>"]
    }
    if path == "/health" {
        give ["status": 200, "type": "application/json",
            "body": json.write(["ok": true, "version": "0.1.0"])]
    }
    if path == "/echo" and method == "POST" {
        give ["status": 200, "type": "text/plain",
            "body": request.get("body") ?? ""]
    }
    give ["status": 404, "type": "text/plain", "body": "no page at {path}"]
}

fn try_it(method: Text, path: Text, body: Text) {
    let reply = handle(["method": method, "path": path, "body": body])
    io.say("{method} {path} -> {reply.get("status")} {reply.get("body")}")
}

fn main() {
    try_it("GET", "/", "")
    try_it("GET", "/health", "")
    try_it("POST", "/echo", "ping")
    try_it("GET", "/nowhere", "")
}
```

Run that and you have tested four routes in a few milliseconds. The same `handle` passed to `http.serve` is the live site.

Building pages

There is no template language. Text interpolation is the template language, and `{{` and `}}` give you literal braces for CSS:

```
use std.io

fn page(title: Text, body: Text) -> Text {
    give "<!doctype html><meta charset=\"utf-8\"><title>{title}</title>" +
        "<style>body{{background:#0b0b0d;color:#f2f2f0;font:16px system-ui}}" +
        "h1{{color:#ffa136}}</style>" + body
}
```

```

}

fn main() {
    io.say(page("Notes", "<h1>Notes</h1><p>Nothing yet.</p>").len().to_text() + " bytes")
}

```

Fetching

```

use std.io
use std.http
use std.json

fn main() {
    let text = http.get("http://example.com/api/items") ?? ""
    let data = json.parse(text)
    io.say("{data}")

    let reply = http.post("http://example.com/api/items", json.write(["name": "pen"]))
    io.say("{reply ?? \"no reply\"}")
}

```

Both give back `nil` when the request fails, so `??` is the natural way to carry on.

A limitation worth knowing. `http.get` and `http.post` speak plain HTTP. There is no TLS, so an `https://` address returns `nil`. Until that lands, fetch through a local proxy or use `sys.run("curl ...")`.

Query strings and encoding

```

use std.io
use std.http

fn main() {
    let q = http.url_encode("tag=work & urgent")
    io.say("encoded {q}")
    io.say("decoded {http.url_decode(q)}")

    var fields: Map<Text, Text> = [:]
    for pair in "tag=work&sort=new".split("&") {
        let bits = pair.split("=")
        if bits.len() == 2 { fields.set(bits[0], http.url_decode(bits[1])) }
    }
    io.say("{fields}")
}

```

The example to read next

`examples/19-web-server.spf` is a complete site: a styled home page, a JSON health endpoint, a clock, and a 404 page, in sixty lines.

```
sprfst run examples/19-web-server.spf
```

Exercise

Add a `/notes` route to the routing example that gives back a JSON list, and a `POST /notes` that adds to it. Keep the notes in a module level `var` and test both by calling `handle` directly.

Learning machines

SPRFST ships the pieces you need to train a small model without reaching for anything else: an array type with the usual operations in `std.tensor`, and layers, a network and a trainer in `std.ai`.

Tensors

A tensor is a shape and a flat list of numbers.

```
use std.io
use std.tensor

fn main() {
  let a = tensor.make([2, 3], [1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
  let b = tensor.make([3, 2], [7.0, 8.0, 9.0, 10.0, 11.0, 12.0])

  io.say("shape      {tensor.shape(a)}")
  io.say("a + a      {tensor.to_list(tensor.add(a, a))}")
  io.say("a * 2      {tensor.to_list(tensor.scale(a, 2.0))}")
  io.say("a @ b      {tensor.to_list(tensor.matmul(a, b))}")
  io.say("a^T       {tensor.to_list(tensor.transpose(a))}")
  io.say("mean      {tensor.mean(a)}")
  io.say("softmax   {tensor.to_list(tensor.softmax(tensor.make([3], [1.0, 2.0, 3.0]))})")
}
  io.say("argmax    {tensor.argmax(tensor.make([4], [0.1, 0.9, 0.3, 0.2]))}")
}
```

The full set: `make zeros random shape at put add sub mul scale matmul transpose relu sigmoid tanh softmax sum mean argmax to_list dot`.

A network

`std.ai` builds on those. A `Dense` layer holds its own weights; a `Network` chains two of them and trains by gradient descent.

```
use std.io
use std.ai
use std.rand
use std.math

fn main() {
  rand.seed(11)

  var net = ai.Network { rate: 0.6 }
  net.build(2, 6, 1)           ~~ 2 inputs, 6 hidden, 1 output

  let inputs = [[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]]
  let wanted = [[0.0], [1.0], [1.0], [0.0]]

  let final_loss = net.train(inputs, wanted, 2000, false)
  io.say("loss after training: {final_loss}")
}
```

```

var correct = 0
for i in 0..4 {
    let got = net.predict(inputs[i])
    if math.round(got[0]) == wanted[i][0] { correct = correct + 1 }
}
io.say("{correct} of 4 correct")
}

```

XOR is the classic test because no single layer can learn it. If the loss stops falling, the usual causes are a learning `rate` that is too large, too few hidden units, or a seed that started somewhere awkward — `rand.seed` makes every run repeatable so you can tell the difference.

The parts

Piece	What it does
<code>ai.Dense</code>	one fully connected layer: start, forward
<code>ai.Network</code>	two layers plus build, predict, learn, train
<code>ai.apply</code> / <code>ai.slope</code>	activations and their derivatives
<code>ai.loss</code>	mean squared error
<code>ai.tokenize</code>	text to word tokens
<code>ai.vocabulary</code>	tokens to numbers

Activations available to a layer: `relu`, `sigmoid`, `tanh`, and `linear` for anything else.

Text in, numbers out

```

use std.io
use std.ai

fn main() {
    let words = ai.tokenize("SPRFST is fast. SPRFST is small. Fast is good!")
    io.say("tokens      {words}")

    let vocab = ai.vocabulary(words)
    io.say("vocabulary {vocab}")

    var encoded: [Int] = []
    for w in words { encoded.push(vocab.get(w) ?? 0) }
    io.say("encoded    {encoded}")
}

```

That is the front of every language model: split, number, feed.

Where the edges are

Be clear eyed about what this is. The gradients in `std.ai` are written out by hand for the two layer case; there is no automatic differentiation, no GPU, and no pretrained weights. It is enough to learn a function from a few thousand examples, to run inference from weights you load yourself, and to understand

exactly what every number is doing—and `std.ai` is ordinary SPRFST source in `std/ai.spf`, so a third layer or a new optimiser is a file you can edit.

Anything larger belongs in a program that calls out to a dedicated runtime, and the extension points in chapter 27 are how you would attach one.

Exercise

Train the network to tell even numbers from odd ones: feed it the four bits of a number from 0 to 15 and expect 1 for even. Then print the cases it gets wrong.

Games

A game is a loop: read what changed, move the world on by a slice of time, draw it. SPRFST gives you the pieces —time in `std.time`, numbers in `std.math`, dice in `std.rand`, pixels in `std.draw`, a window in `app` —and leaves the loop to you.

The shape of the loop

```
use std.time

fn main() {
  var running = true
  var last = time.clock()

  while running {
    let now = time.clock()
    let dt = now - last
    last = now

    update(dt)
    render()

    time.sleep(0.016)      ~~ about sixty frames a second
  }
}
```

`time.clock()` is a monotonic clock in seconds: it never jumps backwards when the system time is corrected, which is what a game loop needs. Passing `dt` into `update` keeps the game running at the same speed whatever the frame rate.

A world that moves

```
use std.io
use std.math

data Vec2 {
  x: Num
  y: Num
}

object Ball {
  pos: Vec2 = Vec2 { x: 20.0, y: 20.0 }
  vel: Vec2 = Vec2 { x: 90.0, y: 60.0 }
  radius: Num = 4.0

  fn step(self, dt: Num, width: Num, height: Num) {
    var x = self.pos.x + self.vel.x * dt
    var y = self.pos.y + self.vel.y * dt
    var dx = self.vel.x
    var dy = self.vel.y
  }
}
```

```

        if x < self.radius { x = self.radius  dx = 0.0 - dx }
        if x > width - self.radius { x = width - self.radius  dx = 0.0 - dx }
        if y < self.radius { y = self.radius  dy = 0.0 - dy }
        if y > height - self.radius { y = height - self.radius  dy = 0.0 - dy }

        self.pos = Vec2 { x: x, y: y }
        self.vel = Vec2 { x: dx, y: dy }
    }
}

fn main() {
    var ball = Ball { }
    let dt = 1.0 / 60.0
    var bounces = 0
    var was = ball.vel.x

    for _frame in 0..240 {
        ball.step(dt, 120.0, 80.0)
        if ball.vel.x != was { bounces = bounces + 1 }
        was = ball.vel.x
    }

    io.say("after 4 seconds the ball is at {math.round(ball.pos.x)}, {math.round(ball.
pos.y)}")
    io.say("it changed direction sideways {bounces} times")
}

```

Two statements on one line are fine when they belong together, as in `x = self.radius dx = 0.0 - dx` — SPRFST ends a statement at the end of a line, not at a semicolon.

Drawing the frame

With the world in variables, drawing is a separate job. In the terminal, a frame is a few lines of text:

```

use std.io

fn frame(bx: Int, by: Int, w: Int, h: Int) -> Text {
    var out = "+" + "-".repeat(w) + "+\n"
    for y in 0..h {
        var row = "|"
        for x in 0..w {
            if x == bx and y == by { row = row + "o" } else { row = row + " " }
        }
        out = out + row + "|\n"
    }
    give out + "+" + "-".repeat(w) + "+"
}

fn main() {
    io.say(frame(3, 2, 20, 6))
    io.say(frame(9, 4, 20, 6))
}

```

On a Mac the same world is drawn into a window with `std.draw` and shown with `app`, or saved as a picture —which is also how you make the title screen:

```
use std.io
use std.draw

fn main() {
  let w = 240
  let h = 160
  let canvas = draw.canvas(w, h)
  draw.clear(canvas, draw.rgb(11, 11, 13))
  draw.rect(canvas, 0, h - 12, w, 12, draw.rgb(28, 26, 24))
  draw.circle(canvas, 70, 90, 10, draw.rgb(255, 161, 54))
  draw.rect(canvas, 150, 60, 10, 46, draw.rgb(120, 200, 255))
  draw.text(canvas, 14, 20, "LEVEL ONE", draw.rgb(242, 242, 240))
  io.say("frame saved: {draw.save_png(canvas, "frame.png")}")
}
```

Dice you can trust

Games need randomness, and tests need the same randomness twice. `rand.seed` fixes the sequence:

```
use std.io
use std.rand

fn roll_three() -> [Int] {
  give [rand.int(1, 7), rand.int(1, 7), rand.int(1, 7)]
}

fn main() {
  rand.seed(2026)
  let first = roll_three()
  rand.seed(2026)
  let again = roll_three()
  io.say("{first} and {again} – the same run twice")

  io.say("a random pick: {rand.choice(["sword", "rope", "lamp"]) ?? "nothing"}")
  io.say("shuffled: {rand.shuffle([1, 2, 3, 4, 5])}")
}
```

State machines beat flags

An `enum` keeps a game honest about what it is doing:

```
use std.io

enum Phase {
  Title
  Playing(Int)
  Paused(Int)
  Over(Int)
}
```

```

fn describe(p: Phase) -> Text {
  give match p {
    when Title -> "press start"
    when Playing(score) -> "playing, score {score}"
    when Paused(score) -> "paused at {score}"
    when Over(score) -> "game over with {score}"
  }
}

fn main() {
  for phase in [Title, Playing(120), Paused(120), Over(340)] {
    io.say(describe(phase))
  }
}

```

The compiler checks that every phase is handled. Add a phase later and it tells you exactly which `match` needs attention.

The example to read next

`examples/20-game.spf` is a complete turn based game: a player object, an event enum, a random encounter table and a win condition.

Exercise

Give the bouncing ball a paddle: a position you move with the keys `a` and `d` read from `io.ask`, one life lost when the ball reaches the bottom, and a score that goes up on every bounce.

Speed

SPRFST is built to start instantly and stay small. This chapter is about seeing where the time goes and what the compiler does about it.

Measuring a run

```
sprfst run main.spf --time
```

prints a line like

```
3 files  parse 1.2ms  check 0.8ms  lower 0.4ms  run 12.6ms  410233 instr  512 KB peak  2 gc
```

Read it from the left: how long the source took to read, to check, to turn into instructions, and then how long your program actually ran, how many instructions it executed, the high water mark of memory, and how many times the collector ran. If `parse` and `check` dominate, the program is small and the figure to care about is startup. If `run` dominates, carry on reading.

Timing part of a program

`time.clock()` is a monotonic clock in seconds. Wrap the part you are suspicious of:

```
use std.io
use std.time

fn slow_text(n: Int) -> Text {
  var out = ""
  for i in 0..n { out = out + "{i}," }
  give out
}

fn quick_text(n: Int) -> Text {
  var parts: [Text] = []
  for i in 0..n { parts.push("{i}") }
  give parts.join(",")
}

fn main() {
  let n = 4000

  let a = time.clock()
  let one = slow_text(n)
  let b = time.clock()
  let two = quick_text(n)
  let c = time.clock()

  io.say("growing text  {(b - a) * 1000.0} ms")
  io.say("list and join {(c - b) * 1000.0} ms")
  io.say("same length: {one.len() >= two.len()}")
}
```

```
}
```

Building text by repeated `+` copies everything it has so far each time. Collecting the pieces and joining once does the copy only once. That single habit is worth more than any compiler flag.

Benchmarks that live with the code

A `bench` block is timed instead of checked. The runner repeats it until the figure settles and reports operations per second.

```
use std.io

fn fib(n: Int) -> Int {
  if n < 2 { give n }
  give fib(n - 1) + fib(n - 2)
}

bench "fib 15" {
  let _ = fib(15)
}

bench "list of 1000" {
  var xs: [Int] = []
  for i in 0..1000 { xs.push(i) }
}

fn main() {
  io.say("fib(15) = {fib(15)}")
}
```

```
sprfst test
```

runs the tests first and then every benchmark.

What the optimiser does

Lowered code goes through five passes, in order:

1. **Constant folding** —arithmetic, comparisons and text joins between literals are worked out once, at compile time.
2. **Move removal** —registers copied onto themselves disappear.
3. **Dead code elimination** —a pure instruction whose result is never read is deleted. Anything that can have an effect is kept.
4. **Jump threading** —a jump to a jump becomes one jump.
5. **Compaction** —the holes left behind are closed up and every jump target is renumbered.

Levels pick how hard it tries:

Flag	Passes	Used by
------	--------	---------

-00	none	debugging, when you want instructions to match source
-01	one round	the default for sprfst run
-02	three rounds	the default for sprfst build
-03	three rounds, same as -O2 today	reserved for the native backend

You can watch it work:

```
sprfst build main.spf -02
```

reports the counts for each pass, and `--ir` writes the instructions to `build/program.spir` where you can read them:

```
built 2 functions, 20 instructions -> ./build/program.spir
opt   folded 2, moves 0, dead 0, jumps 0, nops 0
```

The listing is plain text, one instruction per line, with the source line it came from in the margin. It is the same listing the debugger steps through.

Where the time really goes

A few things that matter more than anything else, roughly in order:

Text built in a loop. Use a list and `join`, as above.

Work inside a loop that could be outside it. Hoist the constant parts out by hand; the optimiser will not move a function call for you because it cannot know the call is pure.

Maps where a list would do. A map lookup hashes; indexing a list does not.

Growing a list one element at a time is fine —it doubles its capacity, so a million pushes is a handful of allocations.

`clone` on a large structure. **It copies. Pass the reference unless** you need an independent copy.

The collector runs between statements and only walks live data. Short lived values are cheap; holding on to everything is not.

Memory

`sys.memory()` gives the bytes the runtime is holding right now, which is useful inside a program:

```
use std.io
use std.sys

fn main() {
  let before = sys.memory()
  var big: [Int] = []
  for i in 0..200000 { big.push(i) }
```

```
let after = sys.memory()
io.say("{big.len()} numbers cost about {(after - before) / 1024} KB")
}
```

A word about the backend

Today `sprfst run` executes the lowered instructions directly in a register machine written in C: no process launch, no JIT warm up, a few milliseconds from typing the command to your first line of output. `sprfst build` writes that same program out as a readable listing rather than a standalone executable. A native code generator is the next thing on the list, and the `-03` level is reserved for it.

Exercise

Write a benchmark that compares looking a word up in a `Map<Text, Int>` against scanning a `[Text]` with `index_of`, for 10, 100 and 1000 entries, and find the size where the map starts to win.

The debugger

`sprfst debug` runs a program under a real debugger: breakpoints that can be conditional, stepping that understands calls, the call stack, every live variable, watches and memory. It is part of the toolchain, not a separate program, so it needs no build flags and no setup.

```
sprfst debug main.spf
```

The commands

Key	Does
r	run to the first breakpoint
c	continue
s	step into
n	step over (next)
o	step out of this function
b <line>	break at a line
b <line> if <cond>	break only when the condition holds
l	list the breakpoints
d <n>	delete breakpoint number n
k	call stack
v	every variable in the current frame
p <name>	print one variable
w <expr>	watch an expression, shown at every stop
m	memory in use, allocations, collections
q	quit
?	the list above

A session

Take a program with a loop:

```
use std.io

fn total(items: [Int]) -> Int {
  var sum = 0
  for n in items {
    sum = sum + n
  }
  give sum
}
```

```

}

fn main() {
    let numbers = [3, 9, 4]
    let answer = total(numbers)
    io.say("total {answer}")
}

```

Break inside the loop, but only for the large value:

```

(debug) b 6 if n > 3
      break 0 at line 6 if n > 3
(debug) r

      breakpoint dbg.spf:6 in total
      4      var sum = 0
      5      for n in items {
      6 >          sum = sum + n
      7      }
      8      give sum
(debug) v
items          [3, 9, 4]
sum            3
n              9
(debug) k
> 2 total dbg.spf:6
1 main  dbg.spf:13
0 <start> dbg.spf:14
(debug) w sum
      watch sum = 3
(debug) c

      breakpoint dbg.spf:6 in total
      watch sum = 12
(debug) q

```

The condition on a breakpoint is ordinary SPRFST, checked in the frame that is stopped, so `n > 3`, `name == "root"` and `items.len() > 100` all work. A condition that cannot be evaluated is treated as false and reported once rather than stopping the program.

Breakpoints land on real lines

Ask to break on a blank line or a comment and the debugger moves the breakpoint forward to the next line that has code, and says so. That is usually what you meant, and it is better than a breakpoint that silently never fires.

Stepping

`s` goes into a call, `n` goes over it, `o` runs until the current function gives back. The listing around the stop always shows the two lines either side, with `>` on the line about to run—not the line that has just finished, which is the usual source of confusion.

Variables you can trust

`v` prints the frame's own locals, in declaration order, with the names you wrote. They survive the optimiser: the debugger runs at `-O0` so the instructions line up with the source exactly. If you want to see what the optimiser did instead, build with `--ir` and read `build/program.spir`.

Memory

`m` reports what the runtime is holding, how many objects are alive and how often the collector has run. It is the quickest way to tell a leak (memory climbing between the same two breakpoints) from a large but stable working set.

When a program crashes

A runtime error prints the same kind of message the compiler does —source line, the value that caused it, and a suggestion —and then the debugger stays open at the point of failure, so `v` and `k` still work.

Exercise

Put a deliberate off by one into the `total` function above, break on the line that gives the answer with a condition that only holds when the sum is wrong, and find it with `k` and `v`.

Forge, and extending SPRFST

Forge is the package manager. It is built into the `sprfst` command, needs no account and no online registry, and keeps every dependency inside your project where you can read it.

A project

```
project.sprfst  what this is, and what it depends on
forge.lock     exactly what was installed, with checksums
src/           your code
tests/         your tests
assets/        files your program loads
packages/      dependencies, copied in
build/         output, safe to delete
```

`project.sprfst` is deliberately dull:

```
[package]
name = "notes"
version = "0.1.0"
entry = "src/main.spf"
description = "A small notebook"

[packages]
mathx = path:../mathx
palette = git:https://example.com/palette.git
```

The commands

Command	What happens
<code>sprfst add <name> --path=../somewhere</code>	copy a local package in and record it
<code>sprfst add <name> --git=<url></code>	clone a package in and record it
<code>sprfst remove <name></code>	delete it from packages/ and both files
<code>sprfst install</code>	make packages/ match project.sprfst
<code>sprfst package</code>	build a .forge archive of this project

Adding a package prints its digest:

```
$ sprfst add mathx --path=../mathx
installed mathx          45c9c4769351
```

and writes `forge.lock` :

```
# Forge lockfile — do not edit by hand
mathx path:../mathx 45c9c4769351aebc70c04263a9920aaa3923f635402bf8ce76ac519cd17a0b7b
```

That digest is a SHA-256 over every file in the package. On the next `sprfst install`, the contents are hashed again and compared. If they differ, the install **stops** rather than quietly using something else. Commit `forge.lock`; it is what makes a build the same for everybody.

Using a package

A package is just a project with modules in it. Import it by name:

```
use std.io
use mathx

fn main() {
    io.say("{mathx.double(21)}")
}
```

The import is resolved in this order:

1. a file beside the one importing it — `shapes.spf`, or `shapes/mod.spf`
2. the entry point named in `packages/<name>/project.sprfst`
3. `packages/<name>/src/<name>.spf`, then `src/main.spf`, then `<name>.spf`
4. the standard library in `$SPRFST_HOME/std/`

So a local file always wins over a package, and a package always wins over nothing. Nothing is fetched behind your back while you are compiling.

Publishing

```
$ sprfst package
packaged ./notes-0.1.0.forge 2 files, 0 KB, 77730369e687
```

A `.forge` file is a plain archive: a `FORGE1` header, one `file <path> <length>` record per file, the bytes, and a SHA-256 of the whole body. Hand it over however you like — a web server, a shared folder, an email. There is no registry to sign up to, and `--git=` means a repository is already a package.

Extending the language

Three doors, from the easiest to the deepest.

1. SPRFST modules

The standard library's own `core`, `collections`, `testing` and `ai` modules are ordinary SPRFST source in `std/`. Anything you write can sit beside them, and `use std.yourthing` will find it. That is the whole extension mechanism for most purposes.

2. Shelling out

`sys.run` gives you the rest of the operating system, and `std.json` makes the conversation easy:

```
use std.io
use std.sys
use std.json
```

```
fn main() {
    let listing = sys.run("ls -1").trim().lines()
    io.say("{listing.len()} entries here")
    io.say(json.write(["cwd_entries": listing.len(), "platform": sys.platform()]))
}
```

3. Native functions

The native table in `compiler/include/sprfst/natives.h` is one line per function:

```
X(MATH_SQRT, "math", "sqrt", "(Num) -> Num", "Square root")
```

The line declares the module, the name, the signature the typechecker enforces and the help text that `sprfst docs` and Studio show. The implementation goes in `compiler/src/nativelib.c` under the matching `NF_` case. Rebuild, and `math.sqrt` exists everywhere—in the checker, the editor's autocomplete, the documentation and the runtime—with no glue code, no bindings file and no registration call.

`extern fn` declarations reserve the same mechanism for functions provided by a host application embedding the runtime.

Editors other than Studio

`sprfst studio` starts a line based service: one JSON request per line, one JSON reply. It answers `check`, `outline`, `tokens`, `complete`, `hover`, `define`, `rename`, `format`, `natives` and `version`. Studio speaks it, and so can any editor you like—it is a dozen lines of glue in most of them.

```
$ echo '{"id":1,"op":"check","file":"src/main.spf"}' | sprfst studio
```

Security, briefly

!22 Packages are copied in, never run during installation. There are no install scripts.

!22 Everything is hashed, and a mismatch stops the build.

!22 `sprfst package` hashes the archive body as well.

!22 The runtime does nothing surprising: no network, no file writes and no process launches unless your program asks for them.

Exercise

Make a package of your own: `sprfst new palette`, give it a module with two or three colour helpers, then `sprfst add palette --path=./palette` from another project and use it. Change one character in the package and watch `sprfst install` refuse.

SPRFST Studio

Studio is the editor that comes with the language. It is a native Mac application —AppKit, Apple Silicon, no web view anywhere —and it drives the same `sprfst` toolchain you have been using from the terminal. Everything it shows you comes from the compiler, so the editor can never disagree with the build.

The window

```
+-----+-----??
?-----+-----??
?----+
| Explorer      | tabs                                | Outline  |
| Problems     |                                     |          |
| Outline      | editor, gutter, minimap                       | Minimap  |
| Git          |                                     |          |
| Debugger     |                                     |          |
+-----+-----??
?-----+-----??
?----+
| Terminal · Run · Playground                        |
+-----+-----??
?-----+-----??
?----+
| project · branch · line:column · timing · memory  |
+-----+-----??
?-----+-----??
?----+
```

Deep black behind, near black panels, warm orange for anything that matters, soft white text. Nothing blinks and nothing bounces.

Shortcuts worth knowing

Keys	Does
Cmd-P	command palette — every command, and every symbol in the file
Cmd-R	run the project
Cmd-B	build
Cmd-U	run the tests
Cmd-D	start the debugger
Cmd-\	toggle a breakpoint on this line
Ctrl-Command	format the file
Ctrl-Command	go to definition
Ctrl-Command	find everywhere
Shift-Command	open a folder

Shift-Command-N	new project
Ctrl-\	focus the terminal
Command-0	open this guidebook inside Studio

The panels

Explorer is the project tree. **Problems** lists every error and warning the compiler reported, with the code (`E0201`), the message and the position; clicking one jumps there. **Outline** lists the modules, types, functions and tests in the current file. **Git** shows the branch, the changed files and a diff, and can stage and commit. **Debugger** has the breakpoint list, the call stack, the variables and the watches, driven by the same engine as `sprfst debug`.

The bottom strip

Terminal is a real shell in the project folder — `sprfst`, `git`, anything. **Run** shows a program's output with its exit code, run time and peak memory. **Playground** runs the current file every time you stop typing and reports the output, the first error, the time it took and the memory it used, which makes it a fast way to try an idea without saving anything.

The editor

Syntax highlighting comes from the compiler's own lexer through the `tokens` request, so a new keyword is coloured the day it exists. Completion, hover and go-to-definition all ask the toolchain as well:

Behaviour	Request behind it
autocomplete	<code>complete <file> <offset></code>
hover documentation	<code>hover <file> <offset></code>
go to definition	<code>define <file> <offset></code>
rename	<code>rename <file> <offset></code>
format on save	<code>format <file></code>
problems	<code>check <file></code>
outline	<code>outline <file></code>

There is a minimap, a breakpoint gutter you can click, inline error marks, and bracket and indentation handling that understands SPRFST blocks.

Fonts

Studio ships four choices, in Settings:

 **SPRFST Hand** —the handwriting face used for the brand, for headings and for the welcome screen, with a readable code face for the code itself

 **Clean Code** —the same code face everywhere

`\225System` —the system interface font

`\225Monospace` —a fixed width face throughout

Size is adjustable, and the choice is remembered between launches.

Running the toolchain yourself

Studio finds `sprfst` in this order: inside its own bundle, then `/usr/local/bin`, then your `PATH`. The status bar shows the version it found. If it cannot find one, every command that needs it says so plainly instead of failing silently.

The protocol is open —`sprfst studio` on the command line starts the same service Studio talks to, one JSON request per line. Chapter 27 lists the requests.

Building Studio

On a Mac with Xcode's command line tools:

```
make app      # builds build/bin/sprfst and dist/SPRFST Studio.app
make dmg      # wraps the app in dist/SPRFST-Studio.dmg
```

`tools/build_macos_app.sh` compiles the Swift sources with `swiftc`, assembles the bundle, writes `Info.plist` with the `.spf` file association, copies the compiler, the standard library, the examples and this guidebook into `Contents/Resources`, and installs the icon. `tools/make_dmg.sh` then stages the app next to a shortcut to `/Applications` and builds a compressed, verified disk image with `hdiutil`.

Both scripts need macOS: `swiftc`, `AppKit` and `hdiutil` exist nowhere else. The compiler, the standard library, the examples and the guidebook build and run on any Unix.

Exercise

Open this repository in Studio, press `Cmd+T` type `guide`, and read chapter one from inside the editor. Then put a breakpoint in `examples/05-functions.spf` and press `Cmd-D`

The standard library

Everything in this chapter arrives with the language. Nothing has to be installed, and nothing reaches the network to work.

The modules

Core

Module	For
<code>std.io</code>	<code>say</code> , <code>print</code> , <code>warn</code> , <code>ask</code> , <code>read_line</code> , <code>flush</code>
<code>std.math</code>	the usual functions, <code>pi</code> , <code>clamp</code> , <code>hypot</code> , <code>is_nan</code>
<code>std.time</code>	<code>now</code> , <code>clock</code> , <code>ms</code> , <code>sleep</code> , <code>format</code> , <code>parts</code>
<code>std.rand</code>	<code>seed</code> , <code>int</code> , <code>num</code> , <code>choice</code> , <code>shuffle</code> , <code>normal</code>
<code>std.sys</code>	<code>args</code> , <code>env</code> , <code>platform</code> , <code>arch</code> , <code>cpu_count</code> , <code>run</code> , <code>memory</code> , <code>exit</code>

Data

Module	For
<code>std.json</code>	<code>parse</code> , <code>write</code> , <code>pretty</code>
<code>std.csv</code>	<code>parse</code> , <code>write</code>
<code>std.hash</code>	<code>sha256</code> , <code>hmac_sha256</code> , <code>crc32</code> , <code>fnv64</code> , <code>base64</code>
<code>std.db</code>	the Ember database — chapter 19
<code>std.tensor</code>	numeric arrays — chapter 23

System

Module	For
<code>std.fs</code>	<code>read</code> , <code>write</code> , <code>append</code> , <code>exists</code> , <code>list</code> , <code>mkdir</code> , <code>size</code> , <code>copy</code>
<code>std.path</code>	<code>join</code> , <code>dirname</code> , <code>basename</code> , <code>ext</code> , <code>abs</code>
<code>std.thread</code>	<code>threads</code> , <code>locks</code> , <code>atomics</code> , <code>channels</code> — chapter 17
<code>std.task</code>	cooperative tasks

Networking, interface, pictures

Module	For
<code>std.net</code>	TCP and UDP sockets
<code>std.http</code>	<code>get</code> , <code>post</code> , <code>serve</code> , <code>url_encode</code> — chapter 22
<code>std.ui</code>	windows and controls — chapter 21

Written in SPRFST, in `std/`

Module	For
<code>std.core</code>	small helpers: <code>between</code> , <code>sign</code> , <code>blend</code> , <code>round_to</code> , <code>times</code> , <code>human_size</code> , <code>human_time</code>
<code>std.collections</code>	<code>zip</code> , <code>unique</code> , <code>chunks</code> , <code>tally</code> , <code>group_by</code> , <code>span</code> , <code>mean</code>
<code>std.testing</code>	<code>same</code> , <code>different</code> , <code>yes</code> , <code>no</code> , <code>close</code> , <code>holds</code> , <code>failed</code>
<code>std.ai</code>	layers, networks, tokenising — chapter 23

Open any of them: they are ordinary source files you can read.

A tour in one program

```
use std.io
use std.core
use std.collections
use std.math
use std.hash
use std.json

fn main() {
  io.say("- core -")
  io.say("between      {core.between(7.0, 1.0, 10.0)}")
  io.say("round_to      {core.round_to(3.14159, 2)}")
  io.say("blend          {core.blend(0.0, 100.0, 0.25)}")
  io.say("human_size     {core.human_size(1536000)}")
  io.say("human_time     {core.human_time(3725.0)}")

  io.say("")
  io.say("- collections -")
  let names = ["ada", "alan", "grace", "edsgar", "grace"]
  io.say("unique          {collections.unique(names)}")
  io.say("chunks          {collections.chunks([1, 2, 3, 4, 5], 2)}")
  io.say("zip              {collections.zip([1, 2, 3], ["a", "b", "c"])}")
  io.say("group_by        {collections.group_by(names, fn(n) => n.slice(0, 1))}")
  io.say("mean            {collections.mean([2.0, 4.0, 9.0])}")

  io.say("")
  io.say("- maths and hashing -")
  io.say("hypot            {math.hypot(3.0, 4.0)}")
  io.say("clamp            {math.clamp(42.0, 0.0, 10.0)}")
  io.say("sha256           {hash.sha256("sprfst").slice(0, 16)}...")
  io.say("base64           {hash.base64("sprfst")}")

  io.say("")
  io.say("- json -")
  let record = ["name": "notes", "version": 1, "tags": ["small", "fast"]]
  io.say(json.write(record))
}
```

Files and paths

```
use std.io
use std.fs
use std.path

fn main() {
    let file = "tour.txt"
    fs.write(file, "one\ntwo\nthree\n")
    fs.append(file, "four\n")

    let text = fs.read(file) ?? ""
    io.say("{text.trim().lines().len()} lines, {fs.size(file)} bytes")
    io.say("name {path.basename(file)} extension {path.ext(file)}")
    io.say("joined {path.join("notes", "today.md")}")

    fs.remove(file)
    io.say("gone: {not fs.exists(file)}")
}
```

Time

```
use std.io
use std.time

fn main() {
    let now = time.now()
    io.say("formatted {time.format(now, "%Y-%m-%d")}")
    let parts = time.parts(now)
    io.say("year {parts.get("year") ?? 0}, month {parts.get("month") ?? 0}")

    let started = time.clock()
    var total = 0
    for i in 0..50000 { total = total + i }
    io.say("counted to {total} in {(time.clock() - started) * 1000.0} ms")
}
```

`time.now()` is wall clock seconds since 1970 —the one to store. `time.clock()` is a monotonic clock — the one to measure with. `time.ms()` is milliseconds since the program started.

Finding out what exists

Three ways, none of which involve a web page:

```
sprfst docs .          # writes docs/ for your project and the library
```

In Studio, hover any name for its signature and its documentation line. And from any editor:

```
echo '{"id":1,"op":"natives"}' | sprfst studio
```

gives the whole native table —module, name, signature and summary —as JSON.

Exercise

Write a `report` program: read a CSV with `std.csv`, group the rows by their first column with `collections.group_by`, print a count per group and the total, and save the summary as JSON next to the input.

A whole program

Everything in this book, used once, on something small and real: a notebook that stores notes in a database, answers from the command line, and comes with its own tests.

Start the project

```
sprfst new notes
cd notes
```

You get:

```
notes/
project.sprfst
src/main.spf
.gitignore
```

sprfst run . already works. Now make it do something.

The program

```
~~ notes — storage, commands and tests in one file.
use std.io
use std.db
use std.sys
use std.time
use std.testing

object Notes {
  store: Int = 0
  next_id: Int = 1

  ~~ `:memory:` for tests, a file path for real use
  fn open(self, place: Text) {
    self.store = db.open(place)
    db.exec(self.store, "create table notes (id int, body text, made num, done int
  )")
    let rows = db.query(self.store, "select id from notes order by id desc limit 1
  ")
    let last = rows.first()?.get("id") ?? 0
    self.next_id = last.to_int() + 1
  }

  fn add(self, body: Text) -> Int {
    let id = self.next_id
    db.exec(self.store, "insert into notes values ({id}, '{body}', {time.now()}, 0
  )")
    self.next_id = id + 1
    give id
  }
}
```

```

fn finish(self, id: Int) -> Bool {
    db.exec(self.store, "update notes set done = 1 where id = {id}")
    give true
}

fn open_notes(self) -> [Map<Text, Any>] {
    give db.query(self.store, "select id, body from notes where done = 0 order by
id")
}

fn count(self) -> Int {
    let rows = db.query(self.store, "select count(*) from notes where done = 0")
    give (rows.first()?.get("count") ?? 0).to_int()
}

fn close(self) {
    db.close(self.store)
}
}

test "notes come back in order and can be finished" {
    var n = Notes { }
    n.open(":memory:")
    let first = n.add("buy milk")
    let second = n.add("write chapter thirty")
    testing.same(first, 1, "first id")
    testing.same(second, 2, "second id")
    testing.same(n.count(), 2, "two open")
    n.finish(first)
    testing.same(n.count(), 1, "one open")
    testing.same(n.open_notes().len(), 1, "one listed")
    n.close()
}

fn main() {
    var notes = Notes { }
    notes.open(":memory:")

    let args = sys.args()
    if args.len() > 0 {
        notes.add(args.join(" "))
    } else {
        notes.add("read the guidebook")
        notes.add("write a program")
        notes.add("show someone")
        notes.finish(2)
    }

    io.say("{notes.count()} notes still open")
    for row in notes.open_notes() {
        io.say(" {row.get("id") ?? 0}. {row.get("body") ?? ""}")
    }
    notes.close()
}

```

Run it:

```
sprfst run .  
sprfst run . -- buy bread
```

Everything after `--` reaches the program through `sys.args()`.

Make it keep the notes

One character: give `open` a file instead of `:memory:`.

```
notes.open("notes.db")
```

Ember writes through on every change, so the notes are there tomorrow. Keep `:memory:` in the test and the test stays fast and isolated.

Check your work

```
sprfst test      # the test block above  
sprfst check    # types, without running anything  
sprfst lint     # style and clarity  
sprfst fmt      # layout, in place  
sprfst docs     # docs/ from your `~~` comments
```

A sensible order before committing: `fmt`, then `check`, then `test`.

Grow it

The shape above is the shape of most programs: a type that owns the data, methods that do one thing each, a `main` that reads the world and prints to it, and tests that never touch the real storage.

From here, the rest of the book plugs straight in.

[\225](#)A window instead of a terminal —chapter 21. The state is `notes`, the list is `list { items: ... }`, and `on click` calls `add`.

[\225](#)A web interface —chapter 22. `handle(request)` reads the path and calls the same methods.

[\225](#)A tidy printout —chapter 6, `pad_right` and `join`.

[\225](#)Tags and grouping —chapter 8 and `collections.group_by`.

[\225](#)Something to share —chapter 27, `sprfst` package.

Where to go next

[\22!](#)`examples/` has twenty one complete programs, from `01-hello.spf` to a neural network that learns XOR.

[\22!](#)`std/` is the part of the library written in SPRFST. Read it; it is meant to be read.

`\22!sprfst docs .` writes documentation for whatever you have written, the same way.

`\225`The language, the compiler, the runtime, Forge, the debugger and Studio are all in this repository, in C and Swift and SPRFST. If something here is wrong, it is fixable, and now you know where.

Final exercise

Take the notes program and finish it properly: `add`, `list`, `done` and `search` as real sub-commands read from `sys.args()`, notes kept in a file, a test for each command, and `sprfst package` at the end. That is a complete piece of software, written in a language that did not exist before this book started.